Rewriting the Structure of Its Own Improvement Mechanism: A Preregistered Causal Test of Bounded Recursive Self-Improvement

Junai P. Felix*
Independent Researcher, Taipei, Taiwan

October 2026
Version 1.1

Abstract

A system that rewrites its own improvement mechanism meets common definitions of recursive self-improvement whether or not the rewriting helps. In five earlier preregistered tests of a Level-4 bounded self-improving program-repair system, the rewritten mechanism was never shown to cause better results than a frozen one. Cheap development measurements located the reason. The rewritable part of the mechanism had only about 1.7 percentage points (pp) to give, while the headroom lay in the form of the policy the mechanism learns, which it could not change.

We made the mechanism's *structure* (how evidence becomes a policy, including the policy's form) a gene that the mechanism's own editing procedure can rewrite. It was offered alongside options known not to help, and we preregistered a causal test. Two independent campaigns proposed rewrites, validated them on fresh seeds and adopted some. One campaign found and adopted, by its own evidence, both parts of the best development setting: a nonlinear, tree-ensemble policy form and a different budget split. It rejected a decoy three times. The other campaign never sampled the useful option.

On 24 fresh confirmatory seeds the evolved mechanisms beat the default by +2.57 pp (95% CI 0.91 to 4.17; 18 of 24 seeds; exact one-sided Wilcoxon $p = 0.0028$). They beat an evidence-free replay of the same proposal process by +3.52 pp ($p = 0.0011$). Every preregistered criterion was met.

The gain came entirely from the campaign that found the structural rewrite: +5.70 pp on 12 of 12 seeds, against -0.57 pp for the other campaign. It was also confined to the training families:

- on four held-out families, the evolved-minus-default difference was +0.26 pp;
- on 164 real bugs it was +0.39 bugs ($p = 0.52$).

A post-hoc leave-families-out analysis shows why. No policy form, linear or nonlinear, transfers more than about 5 pp to unseen families, against a ceiling of 54%. The limit lies in what the system observes, not in its self-modification. For the first time in this programme, rewriting the improvement mechanism paid off causally. The payoff came from a structural change the system selected rather than invented, and it held only in-distribution.

Keywords: recursive self-improvement · self-modification · meta-learning · preregistration · program repair · generalisation

*Correspondence: junai.felixx@gmail.com.

Contents

1 Introduction **3**

2 Background and Related Work **4**

3 Locating the Bottleneck **5**

4 System Under Test **6**
- 4.1 Disclosure boundary 6
- 4.2 Missions 6
- 4.3 The rewritable mechanism 6
- 4.4 The campaign: how Imp rewrites itself 6

5 Design and Preregistration **8**
- 5.1 Arms 8
- 5.2 Criteria 8
- 5.3 Gates 9
- 5.4 What was known before the lock, and a stated risk 9

6 Results I: Gate 0 and the Campaigns **10**
- 6.1 Gate 0 10
- 6.2 The campaigns 10

7 Results II: The Confirmatory Test **12**
- 7.1 What the result is made of 13

8 Results III: Where the Gain Is, and Is Not **14**
- 8.1 Training families versus held-out families 14
- 8.2 Real bugs (S-3) 14
- 8.3 Why the gain does not generalise (post-hoc) 15

9 Discussion **17**

10 Limitations **18**

11 Reproducibility, Integrity and Use of AI Assistance **19**

12 Conclusion **20**

A Per-Seed Results **22**

B Hashes **23**

1 Introduction

Liu et al.'s Level 4 [7], *bounded recursive self-improvement*, asks that a system rewrite its own improvement mechanism. It does not ask that the rewriting help. The Stage 3 study of this programme [1] met every definitional criterion, and four follow-up studies then asked the causal question. Five preregistered tests in all compared an evolved mechanism with the default and with random rewriting on fresh missions. None found a benefit. The latest found harm (-1.12 pp).

Rather than redesign the validator a sixth time, we measured where the system's headroom actually was (Section 3). The answer was simple. Everything the mechanism could rewrite was worth at most about 1.7 pp. The large remaining gaps sat in the *form* of the policy the mechanism learns (about +3–5 pp), its observations and its search, none of which the mechanism could touch. A development gate showed that the system's own learning loop gains +4.65 pp on 8 of 8 seeds when the policy form is a tree ensemble.

This study (v9). We gave the mechanism write access to its own structure. A structural gene now selects how evidence becomes a policy:

- four linear variants, three of which development had shown to be neutral or harmful;
- one nonlinear form.

The same self-editing procedure that edits parameters and its own meta-genes proposes edits to this gene. The same noise-aware validator decides adoption. We preregistered the unchanged causal bars of the previous studies, and we stated in advance the risk that a campaign might never sample the useful option.

Contributions.

1. A layer-by-layer dissection that explains five null tests of productive recursion. The rewritable part of the mechanism had nothing left to give.
2. The first preregistered pass of the causal criteria in this programme: $\mathsf{E} - \mathsf{D}$ +2.57 pp ($p = 0.0028$) and $\mathsf{E} - \mathsf{R}$ +3.52 pp ($p = 0.0011$), with all integrity and autonomy checks.
3. A precise account of what produced it. One campaign adopted the structural rewrite and gained +5.70 pp (12/12 seeds). The other did not sample it and gained nothing. The system captured 45% of the gain of the designer's best setting.
4. A negative result on generality, with its cause. The gain is absent on held-out families (+0.26 pp) and on 164 real bugs (+0.39 bugs). A post-hoc analysis shows that no policy form transfers more than about 5 pp to unseen families: the limit is the system's observations.

2 Background and Related Work

The system. FELIX-C9 is a bounded self-improving program-repair module. A *task policy* decides in which order a fixed repair search tries candidate single-node edits of a faulty function. Each *mission* runs 32 rounds. In every round the system proposes revisions of its own policy from the evidence of its searches, screens them on selection tasks, and promotes one only if it passes a fixed external audit. How evidence becomes revisions is governed by an explicit, versioned improvement mechanism, Imp. Imp also contains the procedure that edits Imp (`meta`), so the mechanism is self-referential. This is the property that meets Liu et al.'s [7] Level-4 definition as operationalised in Stage 3 [1].

Five null tests of productive recursion. Level 4 "as defined" requires that the mechanism be rewritten by itself. It does not require that the rewriting help. Five preregistered studies asked whether it does:

- the Stage 3 confirmatory test;
- a transplant test of evolved mechanisms;
- within-mission validation of rewrites (v4);
- cross-mission validation (v5);
- noise-aware two-stage validation (v6).

None showed that an evolved Imp beats the default Imp on fresh missions. The last was clearly negative: -1.12 pp. A companion study [2] found that the *products* of the improvement loop transfer to real bugs, but again showed no benefit of rewriting the mechanism.

Structural self-modification. Self-referential improvers range from theory to practice:

- the Gödel machine, which rewrites any part of itself once it has proved the rewrite useful [12];
- self-improving code generators that rewrite their own scaffolding [14];
- open-ended archives of self-modifying coding agents [15].

Meta-learning [4] and automated machine learning [5] search over learning algorithms and model classes, in AutoML-Zero's case from primitive operations [11].

Our question is narrower and is asked under preregistration. Given an explicit mechanism whose *structure* (here, the form of the policy it learns) is a rewritable gene alongside its parameters, does the system's own validated self-modification find a structural rewrite that pays off on fresh missions, and does it do better than the same number of rewrites made regardless of evidence?

Gradient-boosted trees. The structural option that development work identified is a gradient-boosted decision-tree ensemble [3], in the histogram form of LightGBM [6] as implemented in scikit-learn [9]. It is an off-the-shelf learner. The system did not invent it. What is tested is whether the system adopts it, by its own evidence, from among options that do not help.

Preregistration. As in previous stages we follow preregistration practice [8, 10]:

- the protocol, criteria, seeds and code were locked and hashed before any campaign or confirmatory mission ran;
- every development run that shaped the design is listed.

3 Locating the Bottleneck

Before building v9 we asked why five redesigns of the validator had failed. Every measurement in this section is exploratory and uses development seeds 2001–2008 only.

Phase 0: what the mechanism can reach. We held each of seven mechanism settings fixed for whole 32-round missions. The settings were the strongest parameter levers found earlier and four structural variants of the linear learner. The best setting was worth +1.68 pp over the default. No setting met the headroom bar (at least +3 pp on at least 6 of 8 seeds). The rewritable part of the system had about 1.7 pp to give, which explains the five nulls.

Dissection: where the headroom is. We then measured ceilings layer by layer, offline, on the exact round-32 test batches (Table 1).

Table 1: Ceilings on the development test batches (mean of 8 seeds, 320 tasks each).

Layer	Solve rate	How measured
Starting policy	27.0%	frozen search, random tie-breaks
Missions' learned policies	38.6%	32-round missions
Best mechanism setting (Phase 0)	40.3%	32-round missions
Best linear policy, oracle labels	33.4–40.1%	offline, 1,500 tasks
Tree ensemble, same features	41.6–43.9%	offline, 1,500–4,000 tasks
Perfect ranking	54.4%	greedy oracle
Any improving path (≤ 4 steps)	64.2%	path oracle

The domain is not saturated. The headroom lies in layers the self-modification could not touch:

- the policy's *form* (linear to nonlinear, about +3–5 pp);
- its observations (about +10 pp);
- the search strategy (about +10 pp).

Gate R0 (offline). With the same data for every model, ten program probes did not help:

- $+1.09$ pp for a linear policy;
- -1.29 pp for trees.

These probes observe the current program's execution without running any candidate. A tree policy beat the best linear policy by +3.36 pp on 8 of 8 seeds.

Gate R1 (missions). The decisive development question was whether the system's *own* loop could exploit the nonlinear form. Here the policy is learned from the mission's own evidence and passes its own promotion and audit, with no oracle labels. The rule was posted before the run: at least +3 pp on at least 6 of 8 seeds.

- Trees alone: +3.59 pp on 5 of 8 seeds. This fails the seed rule.
- Trees with the existing 4×192 budget split: +4.65 pp on 8 of 8 seeds. This passes.

This established that the gain exists and is reachable by the system's loop. It did not establish that the system's self-modification would find it, which is the question of v9.

4 System Under Test

4.1 Disclosure boundary

As in earlier stages, we describe FELIX-C9 at the level of roles and interfaces. We withhold:

- the policy's features and vocabulary;
- how linear revisions are constructed;
- the generator.

Because v9 is about the policy's *form*, we state it. The default policy is linear in its features. The structural option is a gradient-boosted tree ensemble over the same raw features, with hyperparameters drawn from a fixed grid for each candidate.

4.2 Missions

Missions are the v6 code with structural hooks. With the hooks unused, a mission is identical to v6: a v6 Gate 0 mission reproduces its receipt chain byte for byte under the v9 code.

- Each mission trains on 12 development families and is tested on all 16. Four families are held out by a hash of the freeze.
- A mission has 32 rounds. Each round proposes candidates, screens them on selection tasks and promotes at most one through a fixed external audit.

4.3 The rewritable mechanism

Imp has four groups of genes:

- nine parameter gene families: revision-operator mix, regularisation bias, step size, four vocabulary-growth genes, the screening penalty, and the per-round budget split (`alloc`);
- three meta genes, which govern how Imp edits itself;
- **new in v9**, two structural genes:
  - `struct.learner`: how evidence becomes a policy;
  - `struct.blend`: an extra linear revision operator.

Table 2: The learner menu. The initial value is `logistic`, the v6 machinery.

`struct.learner`	Policy form	Development evidence
`logistic` (initial)	linear	baseline
`balanced`	linear (class-balanced)	neutral
`l1`	linear (sparse)	harmful (-1.45 pp)
`pairwise`	linear (ranking loss)	neutral
`trees`	tree ensemble	helps (R0, R1)

4.4 The campaign: how Imp rewrites itself

A campaign runs 12 generations. In each generation the incumbent Imp proposes one variant of itself:

- With probability 0.5, one structural gene is edited. Three times in four this is the learner, switched uniformly to one of the four others; otherwise blend is toggled.
- Otherwise the unchanged v4–v6 operator edits one or two parameter or meta genes, scaled by the incumbent's own `meta.step`.
- A variant is redrawn if it cannot change any mission. In v9 that means its *active part* equals the incumbent's: the genes a mission reads under the variant's learner. Under `trees`, the linear-revision, vocabulary and screening genes are not read. Gate 0 verifies this empirically.

The validator is v6's:

1. **Screen.** The variant runs one mission on each of 4 panel seeds, with common random numbers against the incumbent. It continues only if it solves more round-32 test tasks in total.
2. **Confirm.** The variant and the incumbent each run on 8 fresh seeds that are never reused.
3. **Decide.** Adopt iff the confirmation total exceeds the incumbent's by more than `meta.margin` $\times 8$ tasks. The margin is read from the incumbent and starts at 1.

Two independent campaigns run on separate seed ranges. Every decision is hash-chained and replays from the stored mission records.

5 Design and Preregistration

5.1 Arms

The confirmatory test runs fresh 32-round missions on seeds 7901–7924, which were never used before. Seed s is assigned to campaign $c(s)$ = seed index mod 2. Arms differ only in their Imp.

Table 3: Confirmatory arms.

Arm	Imp
E	Final Imp of campaign $c(s)$
D	Initial Imp (the v6 machinery)
R	Zero-cost replay of campaign $c(s)$: the same proposal procedure, adopting exactly as many edits as E did, at random generations, regardless of evidence
T (reported only)	The designer's best development setting: trees with the 4×192 split

5.2 Criteria

All tests are exact one-sided Wilcoxon signed-rank tests [13] on 24 per-seed paired differences in the round-32 test solve rate.

Table 4: Locked criteria (protocol v9).

ID	Criterion
B-1	Rewriting pays off: E − D mean $\geq$ +1.0 pp and $p < 0.01$
B-2	Evidence matters: E − R mean $\geq$ +0.5 pp and $p < 0.05$
A-1	Promotions in ≥ 20 of 24 E missions
A-2	Each campaign adopted ≥ 1 edit and changed its active part (this is Gate 1)
A-3	Meta is inside Imp and edited by the same procedure
A-4	E second-half progress: mean $\geq$ +1.0 pp and $p < 0.01$
A-5	Integrity: freeze, reconciliation and replay
Headroom	D mean ≤ 0.85, otherwise INVALID
Autonomy	0 interventions; fault suite and broken-supervisor detection

The thresholds of B-1 and B-2 are unchanged from v3–v6 and were not set from development data.

Verdicts.

- *Productive recursion* requires Tier A, B-1 and B-2.
- *Partial* requires Tier A and B-1.
- If B-1 fails, the verdict is *FAIL on the causal question*.

Secondary (S-3). All 96 confirmatory end states and the empty starting policy run on the 164 real bugs of the frozen companion pipeline [2], with budget 12 and 8 tie-break seeds. The contrasts are reported two-sided.

5.3 Gates

Gate 0 requires:

- determinism of default and tree missions, including after a kill and resume;
- a crash-only campaign test;
- headroom;
- the autonomy fault suite;
- inertness of the genes the redraw rule treats as unread under trees.

Gate 1 is A-2. If either campaign adopts nothing, the study stops.

5.4 What was known before the lock, and a stated risk

Known before the lock.

- Phase 0, the dissection, R0 and R1 (Section 3). The designer therefore knew that one menu option helps and that three do not.
- The 0.5 structural share and the tree grid were the designer's choices.

Stated in advance. From the initial `lmp`, a single proposal is the tree learner with probability about 0.11. A campaign therefore samples it at least once in 12 generations with probability about 0.74, and both campaigns do so with probability about 0.55. B-1 could fail simply because a campaign never proposes the useful structure. This was not adjusted.

Not inspected. No campaign proposal stream was inspected before the freeze.

6 Results I: Gate 0 and the Campaigns

6.1 Gate 0

Gate 0 passed on 15 October 2026, after the freeze (`42288423...dedd`) and the registration of the protocol.

Table 5: Gate 0 (all required).

ID	Check	Result
G0-1	Default mission at seed 2902 run twice; tree mission run twice; tree mission killed mid-round 10 and resumed	receipts identical in all three
G0-2	Tiny campaign killed after the screen and mid-confirmation	identical final lmp, log and replay
G0-3	Headroom (default round-32 rate)	44.06% $\leq$ 85%
G0-4	Autonomy fault suite; broken supervisors	14/14 groups; 8/8 detected
G0-5	Inertness: tree mission with all 8 unread genes and blend moved	identical results, weights, spend

Before the freeze, the v9 code reproduced the v6 Gate 0 receipt chain byte for byte. It also reproduced the results of three development tree missions exactly.

6.2 The campaigns

Both campaigns ran 12 generations, from 15 October 15:11Z to 16 October 04:42Z (Figure 1, Table 6).

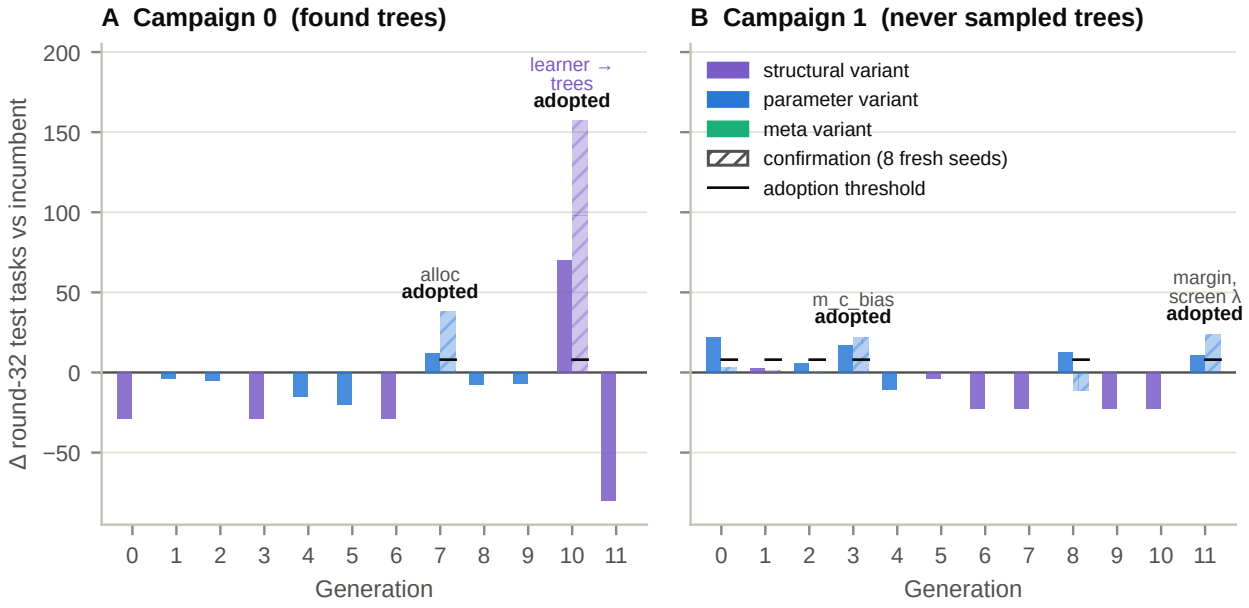


Figure 1: The two campaigns. Solid bars: the variant's screen difference against the incumbent on 4 panel seeds. Hatched bars: the confirmation difference on 8 fresh seeds. Black ticks: the adoption threshold, `meta.margin` $\times$ 8 tasks. **A**: campaign 0 adopted the 4 $\times$ 192 split (generation 7), then the tree policy form (generation 10; confirmation +157 tasks). It rejected the pairwise decoy at generations 0, 3 and 6. **B**: campaign 1's proposal stream never drew the tree learner. It adopted two parameter edits; the second also set its own margin to 0.

Table 6: Campaign record.

	Campaign 0	Campaign 1
Screen passes / adoptions	2 / 2	6 / 2
Adopted edits	`alloc` 1→0 (4 × 192); **learner logistic→trees**	`m_c_bias`; `meta.margin` 1→0 with `screen_lambda` 0→0.77
Decoys rejected	pairwise (×3), and trees→pairwise	l1 (×4), pairwise, blend
Missions (unique)	92 (84)	152 (144)
Reconciled; interventions; replay	84/84; 0; OK	144/144; 0; OK
R (random drift)	generations 4, 11: vocabulary and operator-mix edits	generations 5, 10: blend on, learner→l1

Campaign 0's final Imp is identical, byte for byte, to the designer's setting T. So for campaign-0 seeds E and T are the same condition, and their independently run missions produced identical receipts.

Two session resets killed all processes during campaign 1. The campaign was relaunched each time and continued by crash-only resume, with no change and no intervention.

7 Results II: The Confirmatory Test

Ninety-six missions (4 arms × 24 seeds) ran once on seeds 7901–7924 on 16 October.

Table 7: Preregistered criteria. Means in pp with 95% bootstrap CIs; p is exact one-sided Wilcoxon.

ID	Contrast	Mean	95% CI	$+/-/0$	p
B-1	E − D ($\geq +1.0$, $p < 0.01$)	**+2.57**	[0.91, 4.17]	18/6/0	**0.0028** ✓
B-2	E − R ($\geq +0.5$, $p < 0.05$)	**+3.52**	[1.84, 5.29]	15/7/2	**0.0011** ✓
A-4	E rounds 16→32 ($\geq +1.0$, $p < 0.01$)	+1.60	[0.66, 2.54]	15/5/4	0.0014 ✓
A-1	Promotions in 24/24 E missions ✓				
A-2	Both campaigns adopted 2 edits that changed the active part ✓				
A-3	Meta inside Imp; 1 meta and 1 structural edit adopted ✓				
A-5	Freeze verified; 96/96 and 228/228 missions reconcile; both logs replay ✓				
Headroom	D mean 40.47% $\leq$ 85% ✓				
Autonomy	0 interventions in campaigns and confirmatory ✓				

Verdict: productive recursion (Level 4 with causal support).

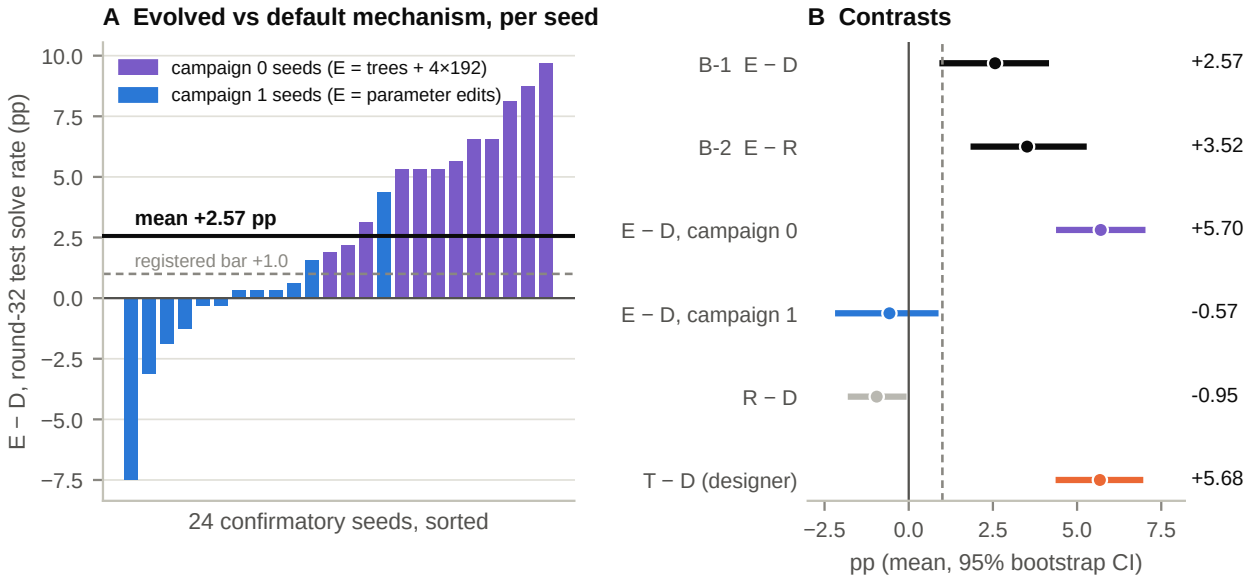


Figure 2: **A**: E − D for each confirmatory seed, coloured by the campaign whose Imp E carries. Every seed with the structural rewrite is positive. **B**: the primary contrasts and their components (95% bootstrap CIs). The dashed line is the registered +1.0 pp bar.

7.1 What the result is made of

Table 8: Reported contrasts (not criteria), pp. p is one-sided unless marked (2).

Contrast	Mean	95% CI	$+/-/0$	p
E − D, campaign-0 seeds (E = trees + 4 × 192)	+5.70	[4.38, 7.03]	12/0/0	0.0002
E − D, campaign-1 seeds (E = parameter edits)	−0.57	[−2.19, 0.89]	6/6/0	0.66
T − D (designer's setting)	+5.68	[4.36, 6.97]	22/2/0	3×10^{-7}
E − T	−3.11	[−4.60, −1.68]	1/11/12	0.001 (2)
R − D	−0.95	[−1.81, −0.07]	8/14/2	0.049 (2)
E − D at round 16	+3.29	[1.72, 4.90]	18/6/0	0.0005

The positive result is the structural rewrite. On the 12 seeds whose E carries it, E beats D on every seed. On the 12 seeds whose E carries only parameter and meta edits, there is no effect, which repeats v6. Because only one campaign sampled the useful option, the system captured 45% of the gain of the designer's setting (T − D +5.68 pp). Random drift with the same number of adoptions was slightly harmful.

Figure 3A shows the arms over the mission. The structural rewrite acts early: by round 16, T is 7 pp ahead of D.

8 Results III: Where the Gain Is, and Is Not

8.1 Training families versus held-out families

Missions train on 12 development families and are tested on all 16. The four held-out families (clamp, floordiv, scaled_sum, span) are never trained on.

Table 9: Gain by family split, pp. p is one-sided for development families and two-sided (2) for held-out families.

Contrast	Mean	95% CI	+/−	p
E − D, development families	+3.33	[1.39, 5.28]	18/4	0.0018
E − D, held-out families	+0.26	[−1.15, 1.61]	11/11	0.62 (2)
T − D, development families	+7.62	[6.13, 9.11]	24/0	6×10^{-8}
T − D, held-out families	−0.16	[−1.98, 1.61]	10/11	0.95 (2)

The tree form's gain exceeds 4 pp on 10 of the 12 development families. It is absent on the held-out families (Figure 3B): two go down, two go up, and the mean is zero. The nonlinear policy fits the regularities of the families it trains on, but it does not carry them to new ones.

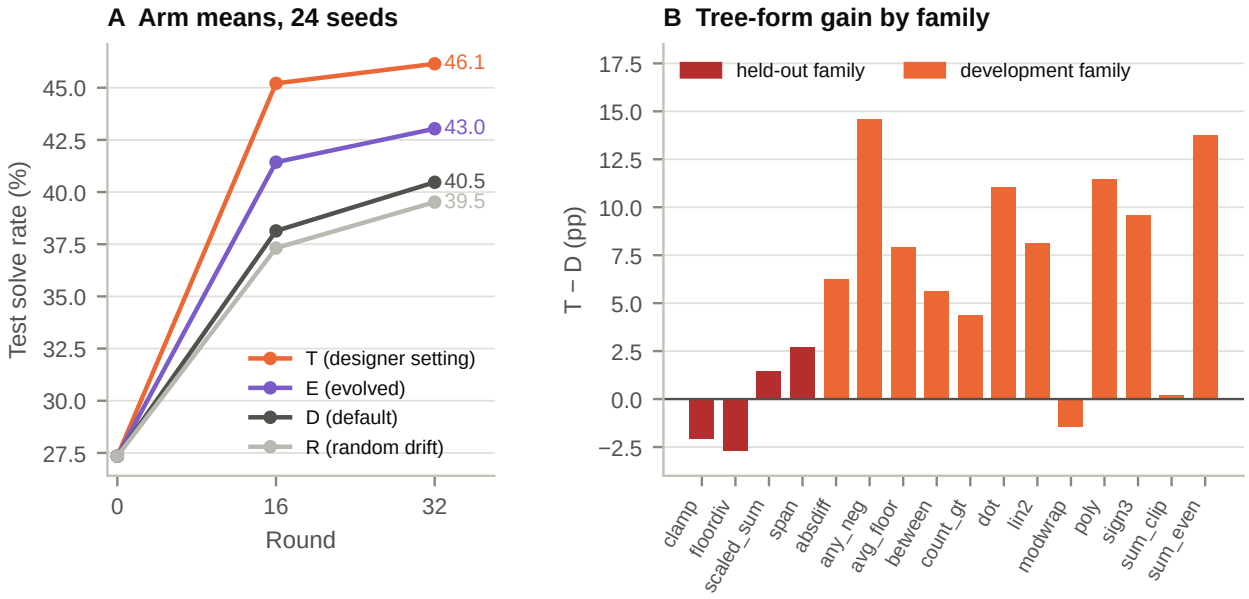


Figure 3: **A**: test solve rate at rounds 0, 16 and 32, mean of 24 seeds. **B**: T − D by family. All of the tree-form gain is on the training families.

8.2 Real bugs (S-3)

All 96 end states and the empty starting policy r0 ran on the 164 valid real bugs (QuixBugs and HumanEvalFix, Python) of the frozen companion pipeline [2]. The budget was 12, with 8 tie-break seeds per task.

Table 10: S-3: real bugs solved (of 164), per-seed means and paired contrasts.

Contrast	Mean	95% CI	$+/-/0$	p
E − D (two-sided)	+0.39	[−0.98, 1.75]	13/11/0	0.52
T − D (two-sided)	+1.06	[−0.34, 2.44]	15/8/1	0.16
E − R (two-sided)	+1.05	[−0.31, 2.46]	17/7/0	0.13
E − r0 (one-sided)	+4.62	[3.57, 5.74]	24/0/0	6×10^{-8}
D − r0 (one-sided)	+4.23	[3.36, 5.10]	23/1/0	1×10^{-7}

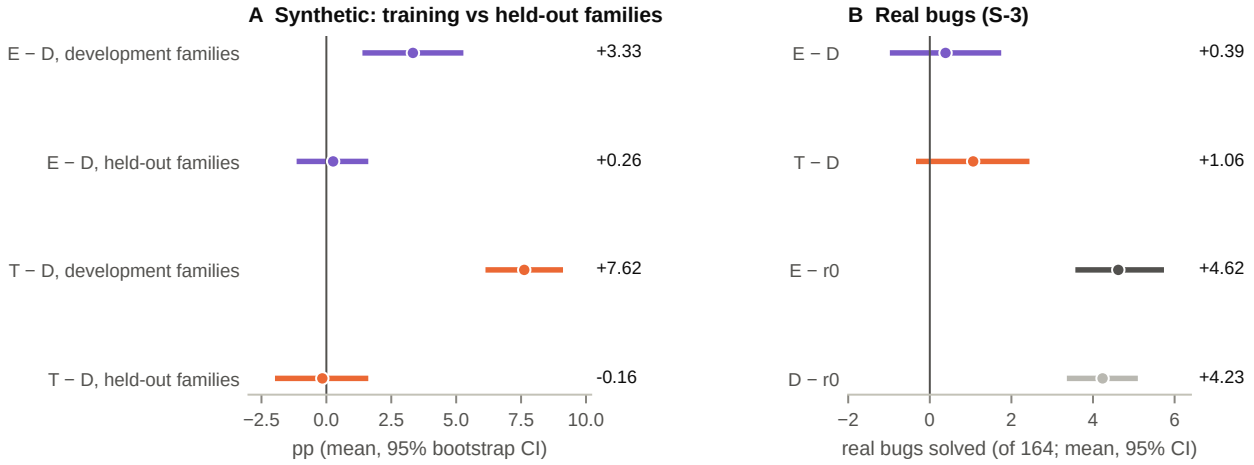


Figure 4: A: the confirmatory gain splits into a large effect on the training families and none on the held-out families. **B**: on real bugs the structural rewrite adds no significant gain. The improvement loop itself transfers again (+4.2 to +4.6 bugs over r0).

The companion study's finding replicates for the third time: what the improvement loop learns transfers to real bugs (+4.2 to +4.6 over r0). The structural rewrite adds no significant real-bug gain beyond that. On campaign-0 seeds, E − D was +1.02 bugs (7 up, 5 down).

8.3 Why the gain does not generalise (post-hoc)

This subsection is post-hoc and exploratory. It uses development seeds and was not part of the preregistration. Two analyses locate the limit.

The same families flip sign. Gate R1 used a different split, holding out dot, lin2, poly and sum_clip. There the tree form gained +7.2 pp on its training families and *lost* 3.1 pp on its held-out families. Three of those families were training families in v9, where the tree form gained on them instead:

- dot: +11.0 pp in v9, against −6.2 pp in R1;
- lin2: +8.1 pp in v9, against −5.0 pp in R1;
- poly: +11.5 pp in v9, against −1.2 pp in R1.

The families are not hard in themselves. The tree form learns structure specific to the families it trains on.

No policy form transfers more. Offline, we trained policies with oracle labels on 12 families and tested them on the 4 left out, in four folds over all 16 families and 8 development seeds (Figure 5).

- On trained families, learned policies reach 38–51%, against 27% for the empty policy.
- On unseen families every reasonable form reaches only 30.6–31.8%, against a perfect-ranking ceiling of 54.4%.
- Linear, tree and hybrid forms tie out of distribution.
- Regularising the trees makes them worse in both settings.

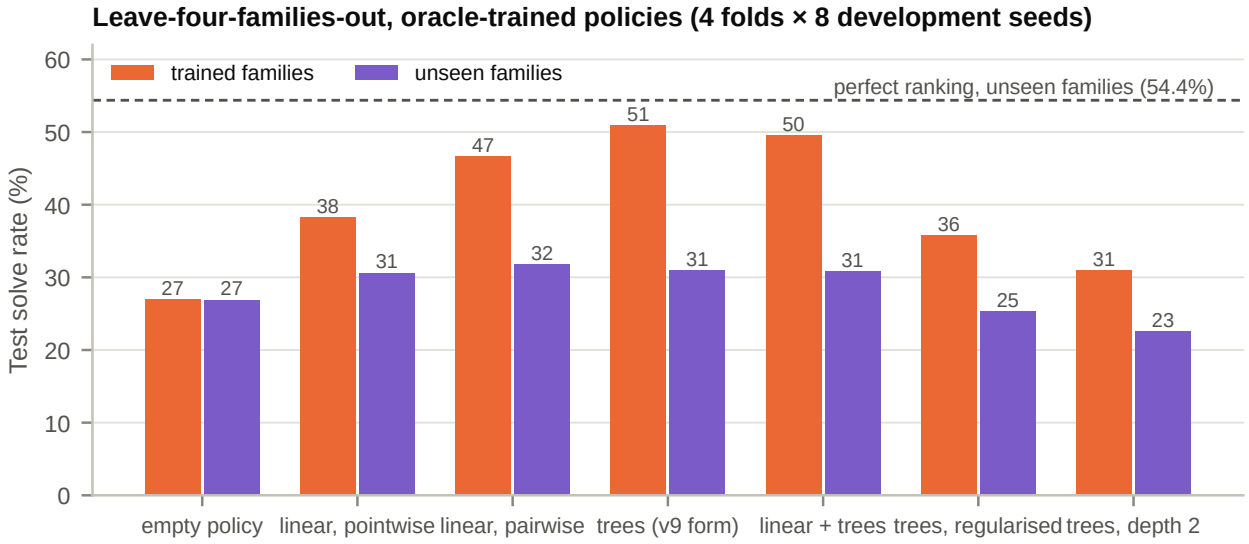


Figure 5: Post-hoc: what transfers to unseen task families. Policy forms differ by up to 25 pp on trained families and by about 1 pp on unseen ones. The gap to perfect ranking on unseen families (dashed) is not closed by any form.

So v9's menu contained no structural rewrite that generalises. The limit lies in what the system *observes*: its candidate features carry little repair knowledge that transfers across families. A different learner cannot fix that; it would need different inputs. The in-distribution scope of the v9 result is therefore a measured property of the system's observations, not a failure of its self-modification.

9 Discussion

What was shown. The causal question of this programme was whether a system that rewrites its own improvement mechanism does better because of the rewriting. For the first time the preregistered answer is yes. The evolved mechanisms beat the default on fresh missions (B-1). They also beat the same number of rewrites made without evidence (B-2), so the gain is due to the system's validation of its own rewrites and not merely to change. The mechanism's own procedure proposed the edit, screened it on 4 seeds, confirmed it on 8 fresh seeds against a margin read from the mechanism itself, and adopted it. No step involved the designer after the freeze.

Why it worked now. Five earlier designs refined the validator. v9 left the validator unchanged and widened what it could act on. The dissection showed the reason. A perfect validator over the old gene space would still have had only about 1.7 pp to find. Once the structure of the learning machinery was writable, the same validator found an edit worth 5.7 pp and adopted it with a confirmation margin twenty times its threshold. Productive recursion here was limited by the reach of self-modification, not by its judgement.

What it does not show. Three qualifications are as important as the result.

1. *Selection, not invention.* The tree form was on a menu the designer wrote, informed by development data. The system found it among decoys and validated it. It did not create it.
2. *One campaign.* The pooled result rests on campaign 0. Campaign 1's proposal stream never drew the tree learner, and its parameter edits were neutral, as in v6. Search over structure was too sparse. Both campaigns sampled the useful option with probability of only about 0.55, and the system captured 45% of the available gain.
3. *In-distribution only.* The gain is on the families the missions train on. It is zero on held-out families and not significant on real bugs. The post-hoc analysis (Section 8.3) shows that this is not specific to trees: no policy form transfers more than about 5 pp to unseen families. The improvement loop's transferable gains, which replicate on real bugs, are the same with and without the structural rewrite.

Level 4 in substance. Earlier stages showed Level 4 "as defined": the mechanism rewrote itself, but the rewrites did not matter. Here the rewrites matter causally, on fresh missions from the training distribution. That is a stronger and more substantive claim. It is still a bounded one: one domain, a designer-supplied menu, and no out-of-distribution benefit. It is not evidence of open-ended or compounding self-improvement, and it is far from Level 5.

Next steps. The post-hoc analysis (Section 8.3) points away from adding structural options over the same observations. The limit to generality is what the system observes, not how it learns from it. Two questions remain open:

- whether different observations would carry repair knowledge that transfers across task families;
- whether structural search can be made reliable across campaigns.

Any such extension warrants the same preregistration, integrity checks and containment as this study.

10 Limitations

- **Designer-supplied options.** The learner menu, the 0.5 structural share and the tree grid were chosen with knowledge of the development results (R0, R1). The decoys were options development had shown to be neutral or harmful. A menu with more, or less obviously separable, options could make the search harder.
- **Same domain for development and test.** R1 used the same task families and mission code, though on different seeds (2001–2008, against 7901–7924). The confirmatory seeds and the campaign seeds were never used in development.
- **Two campaigns.** The result depends on whether a campaign samples the structural option. With two campaigns, one did and one did not. Campaign-to-campaign variance is large and was not estimated beyond that.
- **Generalisation.** No out-of-distribution benefit was found. The post-hoc analysis attributes this to the system's observations. That analysis is exploratory and uses oracle-trained policies, not missions.
- **Arm `T` coincides with `E`** for campaign-0 seeds. `E` and `T` are therefore not independent there; the capture ratio reflects this.
- **Disclosure boundary.** As in earlier stages, the policy's features, the vocabulary and the generator are withheld. The policy form central to this study is disclosed.
- **Single author, AI-assisted.** The study was run by one researcher with substantial AI assistance (Section 11). There has been no independent replication.
- **Infrastructure.** Two session resets interrupted campaign 1. Both were resumed crash-only with no change or intervention, and both are disclosed.

11 Reproducibility, Integrity and Use of AI Assistance

Preregistration and freeze. The protocol was registered before the freeze. It fixes:

- the question, arms, seeds and criteria;
- the design risk;
- what was allowed before the freeze.

The freeze hashes every source file together with the configuration, the initial `Imp`, the campaign settings and the structural settings. Every command verifies the sources against the freeze before running. The freeze record lists every development run. Table 12 gives the hashes.

Determinism and replay.

- Missions are deterministic, including tree missions killed and resumed.
- Tree policies are serialised canonically, so weight hashes do not depend on the process.
- Every campaign decision replays from stored mission records.
- The campaign-0 `E` missions and the `T` missions were run independently and are byte-identical.

Autonomy. No mission in the campaigns or the confirmatory received any intervention. The supervisor fault suite passed 14 of 14 groups and detected 8 of 8 broken supervisors on the frozen build.

Audit package. The frozen code, the run records and an auditor script are available to reviewers on request under a confidentiality agreement. The script verifies the freeze, replays the campaigns and recomputes every reported number.

Author contributions and use of AI assistance. The research direction, the decisions to proceed, publish and disclose, and the system's original design are the author's. The development measurements, the v9 implementation, the execution of all runs, the analysis, the figures and the drafting of this paper were carried out with substantial assistance from an AI system (Claude, Anthropic), under the author's direction. The AI system was not part of the system under test.

12 Conclusion

A self-improving program-repair system, given write access to the structure of its own improvement mechanism, found by its own validated evidence a structural rewrite worth +5.7 pp and adopted it. On fresh confirmatory missions the evolved mechanisms beat the default (+2.57 pp, $p = 0.0028$) and an evidence-free replay (+3.52 pp, $p = 0.0011$). This meets the preregistered criteria for productive recursion that five earlier designs did not.

The result is narrow in three ways:

- it rests on one of two campaigns;
- the rewrite was selected from a supplied menu, not invented;
- it does not extend to held-out families or to real bugs.

The bottleneck for productive self-modification in this system was what the system could rewrite, not how well it judged its rewrites. The next bottleneck is whether what it learns to rewrite generalises.

References

[1] Junai P. Felix. Level-4 bounded recursive self-improvement under a published definition: A preregistered test of a self-referential improvement mechanism. Zenodo, record 23138422, 2026.

[2] Junai P. Felix. Do self-improved repair policies transfer to real bugs? a preregistered external-benchmark test of a level-4 bounded self-improving system. Companion preprint, version 1.1, 2026.

[3] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[4] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(9):5149–5169, 2022.

[5] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren, editors. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[6] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, 2017.

[7] Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, Zheng Wang, Zhihang Fu, and Jieping Ye. The path to recursive self-improving agents: Foundation, framework, and future directions. Preprints.org, preprint 202608.0051, version 1, 2026. Not peer reviewed.

[8] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[9] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[11] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[12] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In Ben Goertzel and Cassio Pennachin, editors, *Artificial General Intelligence*, pages 199–226. Springer, 2007.

[13] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

[14] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024.

[15] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. arXiv preprint arXiv:2505.22954, 2025.

A Per-Seed Results

Table 11: Per-seed round-32 test solve rate (%) and real bugs solved (of 164). Campaign $c(s)$ = seed index mod 2.

Seed	c	E	D	R	T	E−D	Bugs E	Bugs D	Bugs r0
7901	0	40.31	38.44	35.31	40.31	+1.88	23.38	21.12	20.75
7902	1	36.56	36.88	33.75	41.88	-0.31	20.88	22.00	19.62
7903	0	43.12	40.94	36.88	43.12	+2.19	27.38	25.50	22.62
7904	1	42.50	42.19	42.19	49.69	+0.31	26.25	24.75	19.88
7905	0	44.06	37.50	33.75	44.06	+6.56	26.00	23.12	20.75
7906	1	39.38	38.75	39.69	45.31	+0.62	22.12	24.12	20.50
7907	0	48.75	43.44	43.12	48.75	+5.31	27.00	27.12	21.62
7908	1	38.44	38.75	38.75	38.12	-0.31	27.50	26.50	20.50
7909	0	46.25	39.69	35.31	46.25	+6.56	26.38	28.38	20.88
7910	1	45.94	41.56	43.75	50.31	+4.38	21.50	26.88	20.00
7911	0	43.44	37.81	39.38	43.44	+5.62	32.12	26.00	21.88
7912	1	39.69	39.38	40.31	50.94	+0.31	21.00	19.62	19.88
7913	0	45.31	40.00	37.81	45.31	+5.31	29.00	22.12	19.75
7914	1	37.50	35.94	37.81	45.62	+1.56	26.62	26.88	21.38
7915	0	45.31	37.19	40.62	45.31	+8.12	24.75	25.50	21.62
7916	1	35.62	38.75	37.81	41.25	-3.12	30.75	24.88	20.12
7917	0	50.94	41.25	39.38	50.94	+9.69	27.62	25.75	21.38
7918	1	47.19	49.06	47.19	51.88	-1.88	25.12	24.12	20.25
7919	0	45.94	42.81	39.38	45.94	+3.12	27.62	25.00	21.62
7920	1	37.81	39.06	37.81	45.00	-1.25	25.12	22.00	21.00
7921	0	49.06	43.75	41.88	49.06	+5.31	20.38	25.00	18.88
7922	1	41.88	41.56	42.50	50.94	+0.31	22.75	28.50	20.12
7923	0	49.38	40.62	41.56	49.38	+8.75	24.50	29.25	22.25
7924	1	38.44	45.94	42.50	44.69	-7.50	22.88	25.25	20.50
Mean		43.03	40.47	39.52	46.15	+2.57	25.36	24.97	20.74

B Hashes

Table 12: SHA-256 hashes of the frozen sources and run records.

Item	SHA-256
Freeze hash (v9)	`42288423f6b26e5e...2643dedd`
`FREEZE.json`	`bd3f5dbaf7bb93a0...4eb03be6`
Protocol document (as registered)	`9b3d506a54820b67...53d10fe4`
Gate 0 record	`4f5832fcc3b99025...c872bcb9`
Campaign record	`a53f94e793f73bb9...2a0912b0`
Gate 1 record	`8d6a6e87b0cadfeb...41789281`
Confirmatory results	`a91c9e70274a58d4...02b0babb`
S-3 real-bug results	`f4ab23d0dde9fe7a...eeeaa236`
FELIX-C7 pipeline freeze (S-3)	`0b9617b8d2fd774d...ea37aec0`
`autonomy_suite.py`	`c2a8931ab85be250...03598a22`
`campaign9.py`	`791adaf09dfe9037...80fdde2f`
`host.py`	`829e8fa50080739b...52c4d180`
`impmech.py`	`a0360b03abd02572...00f2d7eb`
`levels.py`	`896605b6e40c429c...4677bc71`
`mission.py`	`1a39d023de989f3a...6c206566`
`realbugs9.py`	`710876ffc36c1fd8...2b3ee86b`
`runner9.py`	`a28437769aa3a45c...ce12c323`
`stats.py`	`ee8c288d9207f1de...3cb7d0c6`
`supervisor.py`	`46a9afd5ef2cdb8c...464c401e`